Lec 1 and Lec 2

29th July, 2026

Object-Oriented Programming in Java — Notes

1. Introduction to OOP

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **objects**, which bundle **data (fields/attributes)** and **behavior (methods)** together.

Core idea

Instead of writing a program as a sequence of instructions acting on separate data (procedural style), OOP models real-world entities as objects that interact with each other.

Four Pillars of OOP

Pillar	Meaning
Encapsulation	Bundling data and methods together, hiding internal details
Abstraction	Showing only essential features, hiding complexity
Inheritance	Acquiring properties/behavior of another class
Polymorphism	One interface, many implementations (same method behaves differently)

OOP vs Procedural Programming

Procedural	OOP
Focus on functions	Focus on objects
Data and functions are separate	Data and functions bundled in objects
Harder to reuse code	Reusability via inheritance
Program = sequence of steps	Program = collection of interacting objects
Example: C	Example: Java, C++, Python

---

2. Introduction to Java

Java is a **high-level, object-oriented, platform-independent** programming language developed by **Sun Microsystems** (1995), now owned by **Oracle**.

Key Features

- **Platform Independent** — "Write Once, Run Anywhere" (WORA) via the JVM (Java Virtual Machine)
- **Object-Oriented** — everything (except primitives) is treated as an object
- **Simple & Secure** — no explicit pointers, automatic memory management
- **Robust** — strong type checking, exception handling, garbage collection
- **Multithreaded** — can perform multiple tasks simultaneously

How Java Code Runs

```
Source Code (.java) → Compiler (javac) → Bytecode (.class) → JVM → Machine Code
```

- **JDK (Java Development Kit)** — tools to write & compile Java (includes JRE)
- **JRE (Java Runtime Environment)** — environment to run Java programs (includes JVM)
- **JVM (Java Virtual Machine)** — executes the bytecode; makes Java platform-independent

---

3. Introduction to Classes

A **class** is a **blueprint/template** for creating objects. It defines:

- **Fields** (variables) — the state/data of an object
- **Methods** (functions) — the behavior of an object

An **object** is an **instance** of a class — an actual entity created in memory based on the class blueprint.

Analogy

- Class = architectural blueprint of a house
- Object = the actual house built from that blueprint
- You can build many houses (objects) from one blueprint (class)

```java
class Car {
    // fields
    String color;
```

```
    int speed;

    // method
    void drive() {
        System.out.println("Car is driving");
    }
}
```

---

4. Memory Management in Java

Java memory is mainly divided into two areas:

Memory Area	What it Stores
Stack Memory	Method calls, local variables, references to objects. Follows LIFO order. Automatically cleared when a method finishes
Heap Memory	All objects and instance variables. Shared across the whole application. Managed by the Garbage Collector

Key points

- When you create an object with `new`, the **object itself lives in the Heap**, while the **reference variable** pointing to it lives in the **Stack**.
- **Garbage Collection (GC)** automatically frees heap memory for objects that no longer have any references pointing to them — Java developers don't manually free memory (unlike C/C++).
- Each thread has its own stack; heap is shared by all threads.

```
Car myCar = new Car();
```

- `myCar` (the reference) → stored in **Stack**
- The actual `Car` object (its fields) → stored in **Heap**

---

5. Creating Classes in Java

General/Generic Syntax

```
[access_modifier] class ClassName {
    // fields (data members)
    dataType fieldName;

    // constructor
    ClassName() {
        // initialization code
    }

    // methods (member functions)
    returnType methodName(parameters) {
        // method body
    }
}
```

Example

```
public class Student {
    // fields
    String name;
    int age;

    // constructor
    public Student(String n, int a) {
        name = n;
        age = a;
    }

    // method
    void display() {
        System.out.println(name + " is " + age + " years old.");
    }
}
```

Creating (instantiating) an object

```
Student s1 = new Student("Aarav", 21);
s1.display();
```

- `new` keyword allocates memory on the heap for the object and calls the constructor.

---

6. Encapsulation in Java

Encapsulation = wrapping data (fields) and methods into a single unit (class), and **restricting direct access** to some of the object's components.

How it's implemented

1. Declare fields as `private`
2. Provide **public getter and setter methods** to access/modify them

```
public class Account {
    private double balance;   // hidden from outside

    // getter
    public double getBalance() {
        return balance;
    }

    // setter
    public void setBalance(double amount) {
        if (amount >= 0) {
            balance = amount;
        }
    }
}
```

Why it matters

- **Data hiding** — outside code can't directly corrupt internal state
- **Validation control** — setters can enforce rules (e.g., no negative balance)
- **Flexibility** — internal implementation can change without affecting outside code
- Achieves the "black box" nature of objects

---

7. Functions (Methods) in Java

In Java, functions defined inside a class are called **methods**.

Syntax

```
[access_modifier] [static] returnType methodName(parameterList) {
    // method body
```

```
    return value;  // if returnType isn't void
}
```

Example

```
public int add(int a, int b) {
    return a + b;
}
```

Types of methods

Type	Description
Predefined/Library methods	Already written in Java (e.g., `Math.sqrt()`, `System.out.println()`)
User-defined methods	Written by the programmer
Static methods	Belong to the class itself, not an instance; called via `ClassName.method()`
Instance methods	Belong to an object; called via `objectName.method()`

Method Overloading

Same method name, different parameter lists (a form of compile-time polymorphism):

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
```

---

8. Inheritance in Java

Inheritance allows one class (**subclass/child**) to acquire the fields and methods of another class (**superclass/parent**), promoting code reuse.

Syntax

```
class Parent {
    void greet() {
        System.out.println("Hello from Parent");
    }
}
```

```
class Child extends Parent {
    void message() {
        System.out.println("Hello from Child");
    }
}
```

```
Child c = new Child();
c.greet();    // inherited from Parent
c.message();  // defined in Child
```

Keyword

- `extends` — used by a class to inherit another class
- `super` — used to refer to the parent class (call parent constructor/methods)

Types of Inheritance in Java

Type	Supported?
Single (one parent, one child)	✅
Multilevel (A → B → C)	✅
Hierarchical (one parent, many children)	✅
Multiple (one child, many parents via classes)	❌ (not directly — achieved via **interfaces**)

---

9. Basic Program: Hello World — Line by Line

```
public class HelloWorld{
public static void main(String[]args){
System.out.print("Hello world")
}
}
```

> Note: the code above is missing a semicolon after `System.out.print("Hello world")` — it should be `System.out.print("Hello world");`. Explained below with the correction included.

Line-by-line and word-by-word breakdown

Line 1: `public class HelloWorld {`

Word	Meaning
`public`	Access modifier — this class is accessible from anywhere (any package)
`class`	Keyword to declare a class
`HelloWorld`	Name of the class — must match the filename (`HelloWorld.java`)
`{`	Opens the class body (everything inside belongs to this class)

Line 2: `public static void main(String[] args) {` This is the **entry point** of every Java application — JVM looks specifically for this method signature to start execution.

Word	Meaning
`public`	Access modifier — must be public so the JVM (outside the class) can call it
`static`	Belongs to the class itself, not to any object — so JVM can call it *without creating an object* first
`void`	Return type — this method returns nothing
`main`	The method name — **must be exactly** `main` , this is what JVM looks for
`(String[] args)`	Parameter — an array of Strings, used to accept command-line arguments
`String[]`	Data type: an array of `String` objects
`args`	Name of the parameter (can be any valid identifier)
`{`	Opens the method body

Line 3: `System.out.print("Hello world");`

Word	Meaning
`System`	A built-in final class in `java.lang` package, provides access to system resources
`.`	Dot operator — accesses a member of `System`
`out`	A `static` field of `System` class, of type `PrintStream` — represents the standard output stream (console)
`.`	Dot operator — accesses a member of `out`
`print`	A method of `PrintStream` that prints text **without a new line** (use `println` for a line break after)

Word	Meaning
`("Hello world")`	Argument passed to `print` — the String literal to display
`;`	Statement terminator — **required** in Java after every statement

Line 4: `}` — closes the `main` method body

Line 5: `}` — closes the `HelloWorld` class body

How they interplay

1. JVM loads the `.class` file and looks for a class matching the filename → `HelloWorld`.
2. Inside it, JVM searches for the exact method signature `public static void main(String[] args)` — since it's `static`, JVM can invoke it directly without instantiating `HelloWorld`.
3. Execution begins inside `main`. The single statement calls `System.out.print(...)`, which routes the String to the console via the standard output stream object (`out`).
4. Once the statement finishes, `main` has no more code, so the method ends → the program terminates.
5. The two `{ }` pairs must be **balanced and properly nested**: the method body is enclosed within the class body — this nesting is what makes `main` "belong to" the `HelloWorld` class.

---

10. Default Package in Java

- If a `.java` file **does not declare a package** (no `package` statement at the top), the class belongs to the **default (unnamed) package**.
- Example: the `HelloWorld` class above has no `package` line → it's in the **default package**.

Limitations of default package

- Classes in the default package **cannot be imported** into classes that belong to named packages.
- Not recommended for real projects — used mainly for small/test programs.

With a named package

```
package com.example.myapp;

public class HelloWorld {
```

```
    public static void main(String[] args) {
        System.out.println("Hello world");
    }
}
```

- The `package` statement must be the **first line** of the file (before imports and class declaration).
- Package names conventionally use **reverse domain name** notation and lowercase letters.

---

11. Generic/General Syntax for Class Creation (Summary)

```
[package packageName;]

[import statements;]

[access_modifier] class ClassName [extends SuperClass] [implements Interface]
{

    // fields
    [access_modifier] dataType fieldName;

    // constructor(s)
    [access_modifier] ClassName(parameters) {
        // initialize fields
    }

    // methods
    [access_modifier] [static] returnType methodName(parameters) {
        // logic
        return value;
    }

    // main method (only needed in the class you want to run)
    public static void main(String[] args) {
        ClassName obj = new ClassName(arguments);
        obj.methodName();
    }
}
```

---

12. Taking User Input in Java

Java uses the `Scanner` class (from `java.util` package) to read input from the keyboard.

Steps

1. Import the Scanner class: `import java.util.Scanner;`
2. Create a Scanner object linked to standard input: `Scanner sc = new Scanner(System.in);`
3. Use its methods to read different data types.

Example

```java
import java.util.Scanner;

public class UserInputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println(name + " is " + age + " years old.");

        sc.close(); // good practice — closes the input stream
    }
}
```

Common Scanner methods

Method	Reads
`nextLine()`	A full line of text (String)
`next()`	A single word/token (String, stops at whitespace)
`nextInt()`	An integer
`nextDouble()`	A decimal number
`nextBoolean()`	true/false

> **Gotcha:** Mixing `nextInt()` / `nextDouble()` with `nextLine()` can cause the next `nextLine()` to read an empty string, because the numeric methods don't consume the

> trailing newline character. Fix by adding an extra `sc.nextLine();` after a numeric read if a `nextLine()` follows.

Quick Recap Table

Concept	One-line takeaway
OOP	Paradigm organizing code into objects (data + behavior)
Java	Platform-independent OOP language running on the JVM
Class	Blueprint for objects
Object	Instance of a class, created with `new`
Stack vs Heap	Stack = references/local vars; Heap = actual objects
Encapsulation	`private` fields + public getters/setters
Method	A function defined inside a class
Inheritance	`extends` — child reuses parent's fields/methods
`main` method	JVM's fixed entry point: `public static void main(String[] args)`
Default package	No `package` statement → class sits in the unnamed package
Scanner	Used with `java.util.Scanner` to read user input